Instituto J&F
CENTRO DE EDUCAÇÃO PARA OS NEGÓCIOS

# LAB – DevOps – Pipeline CI/CD

**Aluno:** Noé Carrion | **DevOps–LAB** | **Out/2026**

Enviar em formato PDF no link: https://forms.cloud.microsoft/r/Gbb4U2yXbm

## Atividade — Pipeline CI/CD com IA Generativa

### 1. Informações do Projeto Escolhido

| | |
|---|---|
| **Meu Projeto:** | **Flop+ (FlopPlus)** — Streaming Web App |
| **Meu Repositório no GitHub:** | https://github.com/noecarrion/FlopPlus |
| **Branch que Criei e Enviei:** | `feature/ci-cd-pipeline` |
| **Pull Request no GitHub:** | https://github.com/noecarrion/FlopPlus/pull/new/feature/ci-cd-pipeline |
| **Aplicação Publicada (CD):** | https://noecarrion.github.io/FlopPlus/ |

### 2. Escopo do Deployment e Justificativa da Minha Estratégia

Para atender ao desafio deste laboratório, escolhi o meu projeto de autoria própria **Flop+**, que é uma aplicação web que desenvolvi em **HTML5** e **CSS3** com elementos multimídia (ícones FontAwesome, imagens AVIF/PNG), tratando-se de uma aplicação cliente puramente estática.

Adotei como estratégia de deployment o **GitHub Pages automatizado via GitHub Actions**. Justifico a minha escolha pelas seguintes razões técnicas:

- **Adequação Tecnológica:** Como a minha aplicação é estática front-end, ela não necessita de servidor backend (Node/Python) nem de contêineres Docker rodando continuamente. Utilizar serviços como Render ou Railway traria custos e complexidade desnecessários;
- **Alta Performance e Disponibilidade:** O GitHub Pages entrega o meu site através de uma rede CDN global rápida e segura, com certificado SSL/TLS nativo automático;
- **Segurança com OIDC Nativo:** Configurei o deploy usando as actions oficiais do GitHub (`actions/deploy-pages@v4` e `actions/upload-pages-artifact@v3`), aproveitando as permissões de token nativas do próprio repositório (`pages: write` e `id-token: write`), sem precisar cadastrar chaves de API secretas de serviços externos;
- **Garantia de Qualidade (Quality Gate):** Estruturei a pipeline para que o deployment aconteça automaticamente apenas após um push na branch `main`, e somente se todos os testes e verificações da CI forem concluídos com 100% de sucesso.

### 3. Arquivo da Pipeline que Construí (.github/workflows/ci-cd.yml)

Criei e configurei a pipeline no arquivo `.github/workflows/ci-cd.yml`, dividindo o fluxo em dois jobs integrados e sequenciais:

```
name: Pipeline CI/CD - FlopPlus

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

# Permissões necessárias para publicação no GitHub Pages via OIDC
permissions:
  contents: read
  pages: write
  id-token: write

# Permite apenas uma implantação concorrente no Pages
```

```yaml
concurrency:
  group: 'pages'
  cancel-in-progress: false

jobs:
  # ============================================================
  # ETAPA 1: CONTINUOUS INTEGRATION (CI)
  # Executada a cada push e pull_request direcionados a main
  # ============================================================
  continuous-integration:
    name: CI - Build, Validacao e Testes
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar Ambiente Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: 3. Instalar Dependencias
        run: npm ci

      - name: 4. Executar Testes de Integridade de Links e Assets
        run: npm test

      - name: 5. Executar Validacao Sintatica HTML (HTMLHint)
        run: npm run lint:ci

  # ============================================================
  # ETAPA 2: CONTINUOUS DEPLOYMENT (CD)
  # Executada apenas em push na main, apos conclusao da CI
  # ============================================================
  continuous-deployment:
    name: CD - Deploy no GitHub Pages
    needs: continuous-integration
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar GitHub Pages
        uses: actions/configure-pages@v5

      - name: 3. Fazer Upload do Artefato do Site
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: 4. Realizar Deploy no GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

## 4. Registro do Meu Processo com Apoio da IA Generativa

Durante a criação da pipeline, utilizei a IA generativa como suporte técnico para planejar os estágios, tirar dúvidas práticas de configuração e resolver problemas de execução. Abaixo registro exatamente os prompts que utilizei e as decisões que tomei em cada interação:

### Minha Interação 1 — Definindo a estrutura da pipeline

**O que perguntei:** *"Oi! Preciso criar uma pipeline de CI/CD com GitHub Actions pro meu projeto da faculdade (Flop+). É um site em HTML e CSS no GitHub. O que eu preciso colocar no workflow pra testar e publicar ele no GitHub Pages?"*

**O que obtive:** A IA me explicou a lógica e me sugeriu dividir o fluxo em duas etapas: um job de CI para checar os arquivos e um job de CD para publicar automaticamente no GitHub Pages usando as actions oficiais.

**Problema que notei:** Percebi que o meu repositório original não tinha arquivo `package.json`, testes nem nenhum comando configurado para servir de etapa de validação na CI.

**O que alterei:** Inicializei o projeto com Node.js/npm para criar os scripts de automação e instalei a biblioteca `htmlhint` como dependência de desenvolvimento.

### Minha Interação 2 — Verificando os links do site para o GitHub Pages

**O que perguntei:** *"Eu tenho alguns links nas páginas que estão com barra no começo, tipo href="/index.html" e href="/HTML/aboutme.html". No GitHub Pages isso vai funcionar ou dá problema de link quebrado?"*

**O que obtive:** A IA me alertou que no GitHub Pages o meu repositório roda sob o subcaminho `/FlopPlus/`. Quando um link começa com barra ( `/` ), o navegador tenta acessar a raiz do domínio ( `username.github.io/index.html` ) e quebra a página (erro 404).

**Problema que notei:** Vasculhei o meu código e encontrei exatamente 5 links com barras absolutas em `index.html`, `cadastro.html`, `inicio.html` e `login.html`.

**O que alterei:** Substituí todos os 5 links para usar caminhos relativos ( `../index.html`, `aboutme.html`, `HTML/aboutme.html` ), garantindo que a navegação funcione em qualquer ambiente.

### Minha Interação 3 — Criando um teste real para site estático

**O que perguntei:** *"Como o projeto é só HTML e CSS puro, o professor pediu uma etapa de testes e validação na pipeline de CI. O que eu posso testar de verdade num site estático pra não ficar só um comando vazio?"*

**O que obtive:** A IA me deu a ideia de criar um script em Node.js para percorrer meus arquivos HTML e verificar se todos os arquivos referenciados (CSS, imagens e páginas internas) existem de fato no disco, além de checar a presença das tags fundamentais do HTML e bloquear links absolutos que pudessem quebrar no deploy.

**Problema que notei:** Nenhum; a ideia supriu perfeitamente o critério de validação real exigido no Lab.

**O que alterei:** Escrevi o script `tests/validate-integrity.js`, que roda 44 verificações automatizadas de integridade no projeto.

### Minha Interação 4 — Resolvendo erro de execução no Windows

**O que perguntei:** *"Fui rodar o htmlhint aqui no meu computador pelo terminal do VS Code e deu erro de módulo não encontrado. A pasta do meu projeto tá dentro de 'Instituto J&F', acho que o '&' tá atrapalhando o cmd. Como arrumo o package.json pra rodar sem dar erro aqui no Windows e também funcionar no GitHub Actions?"*

**O que obtive:** A IA me explicou que o caractere `&` é interpretado pelo `cmd.exe` do Windows como separador de comandos, o que fazia falhar o arquivo `.cmd` criado pelo npm na pasta `node_modules/.bin`.

**Problema que notei:** O comando quebrava no meu ambiente local de desenvolvimento no Windows.

**O que alterei:** Configurei o script `lint:ci` no `package.json` invocando o arquivo JavaScript direto pelo Node ( `node node_modules/htmlhint/bin/htmlhint ...` ), rodando redondo tanto no meu Windows quanto no Linux do GitHub Actions.

### Minha Interação 5 — Montando o arquivo final do GitHub Actions

**O que perguntei:** *"Beleza, os testes tão passando. Agora monta o arquivo ci-cd.yml pra mim? Preciso que o teste rode sempre que tiver push ou pull request pra main, e o deploy no GitHub Pages só aconteça se os testes passarem e for direto na main."*

**O que obtive:** A IA me entregou o código do `.github/workflows/ci-cd.yml` estruturado com a dependência `needs: continuous-integration`, a condicional `if: github.ref == 'refs/heads/main' && github.event_name == 'push'`, permissões de OIDC e controle de concorrência.

**Problema que notei:** Nenhum; a estrutura atendeu exatamente ao que o laboratório exigia.

**O que alterei:** Adicionei o arquivo no repositório e incluí comentários explicativos em cada etapa do workflow.

**Minha Interação 6 — Criação de branch, .gitignore e push no Git**

**O que perguntei:** *"Como eu crio uma branch nova chamada feature/ci-cd-pipeline, adiciono um .gitignore pra não subir as pastas do WebStorm e mando essa branch pro meu GitHub?"*

**O que obtive:** A IA me forneceu as regras para o `.gitignore` ignorando `.idea/`, `node_modules/` e saídas de build, e me passou os comandos Git para criar a branch e enviar ao remoto.

**Problema que notei:** A pasta `.idea/` do meu editor estava aparecendo como arquivo não rastreado no Git.

**O que alterei:** Criei o `.gitignore`, criei a branch com `git checkout -b feature/ci-cd-pipeline`, comitei todas as alterações e fiz o push com `git push -u origin feature/ci-cd-pipeline`.

## 5. Minhas Respostas aos Questionamentos Finais do Lab

| Item Avaliado | Minha Resposta |
|---|---|
| **1. Quantos prompts foram necessários para chegar à versão final?** | Utilizei **6 prompts** diretos ao longo da atividade: (1) concepção e divisão dos jobs da pipeline; (2) verificação da compatibilidade de rotas para o GitHub Pages; (3) criação do script de testes em Node.js; (4) solução do bug do caminho com o caractere '&' no Windows; (5) geração do arquivo final `ci-cd.yml`; e (6) criação da branch, configuração do `.gitignore` e envio via Git. |
| **2. Quais foram os principais problemas encontrados?** | 1. **Ausência inicial de testes no projeto:** Como meu projeto continha apenas páginas estáticas, não havia scripts nem ferramentas de validação configuradas;<br>2. **Links com barra absoluta no HTML:** Identifiquei 5 links que usavam barra na raiz (ex: `/index.html`), o que quebraria no GitHub Pages por ele rodar dentro da subpasta `/FlopPlus/`;<br>3. **Conflito do caractere '&' no meu Windows:** Como a pasta do projeto estava em "Instituto J&F", o `cmd.exe` falhava ao rodar os executáveis `.cmd` gerados pelo npm;<br>4. **Arquivos de editor sem ignorar:** A pasta interna `.idea/` do meu WebStorm estava solta no Git sem um `.gitignore`. |
| **3. Quais alterações foram necessárias?** | • Criei o arquivo de workflow `.github/workflows/ci-cd.yml` com os jobs de CI e CD;<br>• Criei o `package.json` com os comandos `test`, `lint:ci` e a dependência do `htmlhint`;<br>• Desenvolvi o script `tests/validate-integrity.js` com 44 verificações automatizadas;<br>• Corrigi os 5 links absolutos nos arquivos HTML para caminhos relativos;<br>• Criei o arquivo `.gitignore` para proteger o repositório. |
| **4. Quais alterações foram realizadas manualmente?** | • Naveguei manualmente no navegador pelas páginas do site após as alterações para confirmar que o visual, imagens e botões continuavam funcionando sem regressões;<br>• Validei minhas credenciais do Git no Windows para liberar o envio do push ao repositório remoto;<br>• Defini o nome da branch de entrega ( `feature/ci-cd-pipeline` ). |
| **5. Como você verificou que a pipeline estava funcionando corretamente?** | 1. **Testes locais no terminal:** Rodei `npm test` (todas as 44 verificações passaram com êxito) e `npm run lint:ci` (0 erros no HTML);<br>2. **Validação do arquivo YAML:** Conferi a sintaxe dos gatilhos, a dependência `needs: continuous-integration` e a condição de push na branch `main`;<br>3. **Confirmação no GitHub:** Enviei a branch `feature/ci-cd-pipeline` para o GitHub com sucesso, gerando o link de Pull Request oficial;<br>4. **Conferência do fluxo CI → CD:** Garanti que o job de deployment só é acionado se a etapa de CI terminar com status de sucesso na branch principal. |

## 6. Minha Análise do Processo de Desenvolvimento com IA

Utilizar a IA generativa durante a atividade acelerou bastante a minha produtividade na construção da pipeline de CI/CD. Em vez de ficar procurando exemplos genéricos na documentação, consegui fazer perguntas pontuais sobre a realidade do meu

projeto, o que me ajudou a antecipar armadilhas comuns, como a quebra de links no GitHub Pages por causa do subdiretório do repositório.

Por outro lado, pude constatar que a minha atuação crítica foi indispensável: quando o comando do linter quebrou no meu computador por conta do caractere especial `&` no caminho da pasta, precisei analisar o erro do terminal, explicar o cenário para a IA e validar a adaptação do script para garantir que ele funcionasse tanto no meu ambiente Windows quanto no runner Linux do GitHub Actions. Tratei a IA como uma parceira de programação em pares, onde coube a mim tomar as decisões de arquitetura, testar as soluções e homologar o resultado final.

## 7. Minhas Evidências de Execução da Pipeline CI/CD

**Evidência 1: Testes que executei localmente no terminal (npm test e npm run lint:ci)**

```
> flopplus@1.0.0 test
> node tests/validate-integrity.js

==================================================
  FLOP+ - TESTE DE INTEGRIDADE E VALIDACAO DE ASSETS
==================================================

[1/3] Verificando arquivos essenciais da aplicacao...
  ✓ index.html encontrado.
  ✓ LICENSE encontrado.
  ✓ CSS\style.css encontrado.
  ✓ IMG\logo.png encontrado.
  ✓ IMG\background.avif encontrado.
  ✓ index.html encontrado.
  ✓ HTML\aboutme.html encontrado.
  ✓ HTML\cadastro.html encontrado.
  ✓ HTML\inicio.html encontrado.
  ✓ HTML\login.html encontrado.

[2/3] Verificando estrutura e doctype dos arquivos HTML...
  ✓ Estrutura basica de index.html validada.
  ✓ Estrutura basica de HTML\aboutme.html validada.
  ✓ Estrutura basica de HTML\cadastro.html validada.
  ✓ Estrutura basica de HTML\inicio.html validada.
  ✓ Estrutura basica de HTML\login.html validada.

[3/3] Validando links (href) e assets (src)...

==================================================
TOTAL DE VERIFICACOES REALIZADAS: 44
SUCESSO: Todos os arquivos, estruturas e links estao integros!
==================================================

> flopplus@1.0.0 lint:ci
> node node_modules/htmlhint/bin/htmlhint index.html HTML/*.html

Scanned 5 files, no errors found (28 ms).
```

**Evidência 2: Criação da Branch e Push Remoto que realizei para o GitHub**

```
$ git checkout -b feature/ci-cd-pipeline
Switched to a new branch 'feature/ci-cd-pipeline'

$ git add .gitignore package.json package-lock.json tests/ .github/ HTML/ index.html
$ git commit -m "feat: implementa pipeline de CI/CD com GitHub Actions e GitHub Pages"
[feature/ci-cd-pipeline b9f6f7a] feat: implementa pipeline de CI/CD com GitHub Actions e GitHub P
ages
 9 files changed, 605 insertions(+), 5 deletions(-)

$ git push -u origin feature/ci-cd-pipeline
branch 'feature/ci-cd-pipeline' set up to track 'origin/feature/ci-cd-pipeline'.
To https://github.com/noecarrion/FlopPlus.git
 * [new branch]      feature/ci-cd-pipeline -> feature/ci-cd-pipeline
```

**Instruções para Demonstração e Publicação no GitHub**

1. No repositório do meu projeto (github.com/noecarrion/FlopPlus), acessei **Settings > Pages**;
2. Em **Build and deployment > Source**, selecionei a opção **"GitHub Actions"**;
3. Abri o **Pull Request** da branch `feature/ci-cd-pipeline` para a branch `main` pelo link: Criar Pull Request;
4. Na aba **Actions**, acompanhei a execução automática do job de validação (CI);
5. Ao aprovar e realizar o merge para a branch `main`, o job de CD é acionado automaticamente e publica a aplicação na URL: https://noecarrion.github.io/FlopPlus/.

---